

Query Optimization: Q & A

1 Algebraic Transformations

Q1: Logical vs. Physical Equivalence

Explain the difference between logical equivalence and physical equivalence of relational-algebra expressions.

Solution

- **Logical Equivalence:** Two expressions are logically equivalent if they generate the same set (or multiset) of tuples on every legal database instance. This is used in the transformation phase of optimization.
- **Physical Equivalence:** Two expressions are physically equivalent if they are logically equivalent AND produce the same "interesting sort order" (e.g., both output rows sorted by ID). This is crucial for choosing physical operators like Merge Join.

Q2: Selectivity and Pushing Selections

Why is it usually beneficial to "push" selections as far down the query tree as possible? Are there any cases where it could be detrimental?

Solution

Benefits: Pushing selections reduces the number of tuples early in the pipeline. This reduces the cost of subsequent joins and sorting, as they process smaller data volumes.

Detriments: Rarely detrimental, but if a selection condition is extremely expensive to evaluate (e.g., a complex user-defined function) and subsequent operations have very high selectivity anyway, it might be better to defer the selection until the tuple count is even smaller.

2 Cost Estimation and Statistics

Q3: Histogram Comparison

Compare equi-width and equi-depth histograms. Why is equi-depth often preferred in practice?

Solution

- **Equi-width:** Divides values into blocks of equal range (e.g., 0-10, 11-20). It suffers if data is heavily skewed (some buckets may be empty while others are massive).
- **Equi-depth:** Adjusts boundaries so each bucket has the same number of tuples.

Preference: Equi-depth provides much higher resolution for "hot" ranges of data, leading to more accurate size estimations for skewed datasets.

Q4: Join Size Estimation Logic

If $R \cap S$ is a foreign key in S referencing R , what is the estimated size of $r \bowtie s$? Explain the reasoning.

Solution

The estimated size is exactly n_s .

Reasoning: Every tuple in s has a foreign key that matches exactly one primary key in r . Since it is a foreign key, it is guaranteed that a matching tuple exists in r . Thus, every tuple in s participates in the join exactly once.

3 Choice of Evaluation Plans

Q5: Join Order Complexity

What is the time complexity of finding the best join order for n relations using dynamic programming for (a) Bushy trees and (b) Left-deep trees?

Solution

- **Bushy Trees:** $O(3^n)$. The optimizer considers all subsets and all their possible splits.
- **Left-Deep Trees:** $O(n2^n)$. The search space is restricted to adding one relation at a time to a previously computed join result.

4 Nested Subqueries

Q6: Correlated Evaluation vs. Decorrelation

Why is correlated evaluation considered inefficient, and how do semijoins help?

Solution

Inefficiency: In correlated evaluation, the subquery is invoked for every row of the outer query. This leads to thousands of small queries and heavy **random I/O**.

Semijoin Role: Decorrelation transforms the subquery into a semijoin ($\bowtie$). This allows the optimizer to use **Set-oriented** join algorithms (like Hash Join) which process data in large, sequential batches, drastically reducing I/O.

Q7: The Duplicate Problem

Why can't we simply replace an EXISTS subquery with a standard natural join $\Pi_{name}(r \bowtie s)$?

Solution

Because SQL uses **multiset semantics**. If a tuple in r matches multiple tuples in s , a standard natural join will produce multiple copies of the r tuple in the output. EXISTS in SQL only checks for presence and does not duplicate the outer rows. A **multiset semijoin** ($\bowtie$) is required to preserve the correct duplicate count.

5 Materialized Views

Q8: Incremental Join Maintenance

Show how to incrementally update the materialized view $v = r \bowtie s$ when a set of tuples i_r is inserted into r .

Solution

Using the distributive property of join over union:

$$v_{new} = (r_{old} \cup i_r) \bowtie s = (r_{old} \bowtie s) \cup (i_r \bowtie s)$$

Since $v_{old} = r_{old} \bowtie s$, we have:

$$v_{new} = v_{old} \cup (i_r \bowtie s)$$

We only need to compute the join for the **new incoming tuples** i_r and add the results to the stored view.

Q9: View Maintenance for Aggregates

Why is it necessary to maintain a tuple count for a **sum** aggregation in a materialized view?

Solution

Without a count, we cannot distinguish between:

1. A group where the current sum of values is 0.
2. A group that has become empty because all its tuples were deleted.

In the second case, the group entry *must* be removed from the view. The count tells us exactly when the last tuple is gone.

6 Advanced Algebraic Equivalences

Q10: Join and Set Difference

Show that the following equivalence holds:

$$E_1 \bowtie_{\theta} (E_2 - E_3) \equiv (E_1 \bowtie_{\theta} E_2) - (E_1 \bowtie_{\theta} E_3)$$

Explain why this rewrite might improve efficiency.

Solution

Proof Sketch:

- Let t be a tuple in the LHS result. Then t prefix from E_1 matches some t' in $E_2 - E_3$. This means t' is in E_2 but not in E_3 .
- Thus, t is in $(E_1 \bowtie E_2)$ but cannot be in $(E_1 \bowtie E_3)$, which matches the RHS.

Efficiency: Evaluating the RHS might be helpful if most tuples in E_2 are also in E_3 , as the subtraction reduces the final result size. However, the LHS is usually better if $E_2 - E_3$ can be computed very quickly, avoiding the extra join with E_1 .

Q11: Pushing Aggregation through Join

Under what conditions can the following be equivalent?

$$A, B \gamma_{agg(C)}(E_1 \bowtie E_2) \equiv (A \gamma_{agg(C)}(E_1)) \bowtie E_2$$

Solution

This equivalence holds if:

1. E_2 contains only attributes A and B .
2. A is the primary key for E_2 .
3. Every tuple in E_1 has a matching tuple in E_2 .

If E_2 has duplicate values for A , the aggregate results on the LHS would count those duplicates, while the RHS (doing aggregation before join) would not.

7 Advanced Size Estimation

Q12: Join Size with Primary Keys

Consider relations $r_1(A, B, C)$, $r_2(C, D, E)$, and $r_3(E, F)$ with primary keys A , C , and E respectively. Assume $n_1 = 1000$, $n_2 = 1500$, $n_3 = 750$. Estimate the size of $r_1 \bowtie r_2 \bowtie r_3$.

Solution

- **Step 1 ($r_1 \bowtie r_2$):** Since C is the primary key of r_2 , each tuple in r_1 joins with at most 1 tuple in r_2 . Size ≤ 1000 .
- **Step 2 ($(r_1 \bowtie r_2) \bowtie r_3$):** Since E is the primary key of r_3 , each tuple in the previous result joins with at most 1 tuple in r_3 .
- **Final Result:** The size is ≤ 1000 .

8 Indexing and Negation

Q13: Negation in B+ Trees

A B+ tree index is available on building for relation department. How should the following be handled?

$$\sigma_{\neg(\text{building} < \text{'Watson'})}(\text{department})$$

Solution

The query is logically equivalent to $\sigma_{\text{building} \geq \text{'Watson'}}(\text{department})$. Since we have a B+ tree index, it points to the first tuple where $\text{building} \geq \text{'Watson'}$. The system can then follow the leaf node pointers to retrieve all subsequent tuples in the sequence.

9 Optimization Complexity

Q14: Catalan Number and Join Orders

Show that for n relations, there are $\frac{(2(n-1))!}{(n-1)!}$ different join orders.

Solution

Each join order is represented as a complete binary tree with n leaf nodes (the relations). The number of such trees is given by the **Catalan Number** $C_{n-1} = \frac{1}{n} \binom{2n-2}{n-1}$. Multiplying this by the $n!$ possible permutations of the relations at the leaves gives the total number of labeled join orders.

10 Complex Decorrelation

Q15: Scalar Aggregate Decorrelation (SQL)

How can the following correlated subquery be decorrelated into a non-nested SQL query?

```
select S.acc_no from account S
where S.balance = (select max(T.balance) from account T
                  where T.branch = S.branch);
```

Solution

Step 1: Compute Group Maxima:

```
create table temp1 as
select branch, max(balance) as m_bal
from account group by branch;
```

Step 2: Join with Original Relation:

```
select account.acc_no from account, temp1
where account.branch = temp1.branch
and account.balance = temp1.m_bal;
```

11 Logical Non-Equivalence

Q16: Counter-examples in Relational Algebra

For each pair, show they are NOT equivalent using instances:

- $\Pi_A(r - s)$ and $\Pi_A(r) - \Pi_A(s)$
- $\sigma_{B < 4}(A \gamma_{\max(B)}(r))$ and $A \gamma_{\max(B)}(\sigma_{B < 4}(r))$

Solution

a. $r = \{(1, 2)\}, s = \{(1, 3)\}$:

- LHS: $\Pi_A(\{(1, 2)\}) = \{1\}$.
- RHS: $\Pi_A(r) - \Pi_A(s) = \{1\} - \{1\} = \emptyset$.

b. $r = \{(1, 2), (1, 5)\}$:

- LHS: $\sigma_{B < 4}(\text{max is } 5) = \emptyset$.
- RHS: $\gamma_{\max}(\{1, 2\}) = 2$. Output is $\{(1, 2)\}$.

12 Estimating Sizes without Keys

Q17: General Join Size Estimation

Consider $r_1(A, B, C), r_2(C, D, E), r_3(E, F)$. No primary keys.

$V(C, r_2) = 1100, V(E, r_3) = 100$. $n_1 = 1000, n_2 = 1500, n_3 = 750$. Estimate size.

Solution

- **Join** $r_1 \bowtie r_2$: Average tuples per r_1 row is $n_2/V(C, r_2) = 1500/1100 = 15/11$.
- Intermediate Size $n_{12} = 1000 \cdot (15/11) = 15000/11 \approx 1363.6$.
- **Join with** r_3 : $n_{123} = n_{12} \cdot (n_3/V(E, r_3)) = (15000/11) \cdot (750/100)$.
- **Final Size**: $\approx 10,227$ tuples.

13 Advanced Optimization Scenarios

Q18: Joins on Computed Functions

How can a database optimize a join with a condition like $\text{upper}(r.A) = \text{upper}(s.A)$?

Solution

- **Hash Join**: Hashing must be performed on the *result* of the $\text{upper}()$ function rather than the raw attribute values.
- **Merge Join**: The input relations must be sorted based on $\text{upper}(A)$.
- **Index**: A **Functional Index** (or Index on Expression) on $\text{upper}(A)$ would allow for an Index Nested-Loop Join.

Q19: Formal Definition of Interesting Orders

What constitutes an "interesting order" for a subset of relations S_1 within a larger query S ?

Solution

An order is interesting if it is a sort order on any subset of attributes T , where T contains attributes of S_1 that participate in join conditions with relations in $S - S_1$. Preserving such orders can make future joins (like Merge Join) or grouping operations significantly cheaper.

Q20: General Procedure for Decorrelation

Provide a general procedure for decorrelating a query of the form:

```
select L1 from r where P1 and A1 op (select f(A2) from s where P2)
```

Solution

1. **Separate Predicates:** Split P_2 into P_{21} (independent) and P_{22} (correlated).
2. **Create Grouped Subquery:** Compute a temporary relation T_1 by grouping s on the attributes used in P_{22} , applying $f(A_2)$.
3. **Final Join:** Perform a natural join between r and T_1 on the correlation attributes, adding the condition $r.A1 \text{ op } T1.f_result$.

These exercises illustrate the rigorous mathematical foundation required for efficient query processing.